

# EGE UNIVERSITY

Faculty of Engineering • Department of Computer Engineering  
*Computational Intelligence and Deep Learning — 2025/2026 Spring Term*

## VOLTOPTIMIZER

### A Hybrid 1D-CNN + GRU Based RUL Estimation and Multi-Agent Decision Support System for EV Batteries

*A Hybrid 1D-CNN + GRU Based RUL Estimation and Multi-Agent Decision Support System for EV Batteries*

---

|                 |                                              |
|-----------------|----------------------------------------------|
| PROJECT NAME    | CIDL-2026-P2-VOLTOPTIMIZER                   |
| COURSE NAME     | Computational Intelligence And Deep Learning |
| PROJECT NO      | Project 2 (Hybrid DL + Agentic AI)           |
| ACADEMIC TERM   | 2025 / 2026 Spring                           |
| SUBMISSION DATE | June 2026                                    |

---

#### Authors

Erdem Karabil — 05220000365  
Ömer Faruk Şaylan — 05220000360

Department of Computer Engineering  
Ege University, İzmir, Turkey

## Abstract

---

With the rapid proliferation of electric vehicles, Remaining Useful Life (RUL) estimation for Battery Management Systems (BMS) has become a critical problem from both safety and economic perspectives. This study proposes VoltOptimizer, an integrated system that combines a hybrid 1D-CNN + GRU + Attention deep learning model for RUL prediction with a three-agent Agentic AI system for real-world decision support.

The proposed hybrid model extracts short-term temporal patterns via 1D convolutional layers, captures long-term dependencies through multi-layer GRU, and highlights critical time steps using a lightweight attention mechanism. The model is trained on five-feature synthetic time-series windows (cell voltage, current, temperature, vehicle speed, road slope) and achieves Test MSE = 8.54, MAE = 2.40 (RUL %), and  $R^2 = 0.9875$  with early stopping at around epoch 27.

In the multi-agent system, Battery Guardian Agent dynamically adjusts safety parameters using the RUL prediction and real-time battery data; Grid Tariff Agent identifies the cheapest charging windows from a simulated grid API; and Smart Trip Agent produces a holistic travel plan by combining route, tariff, and safety constraints via the RoutePlannerTool. The orchestrator merges all three agents' outputs into a single decision report.

*Keywords: Deep learning, 1D-CNN, GRU, attention mechanism, battery RUL estimation, electric vehicle, multi-agent systems, Agentic AI, smart grid.*

## 1. Introduction

---

Electric vehicles (EVs) have been spreading rapidly in recent years, driven by environmental sustainability policies and advances in battery technology. However, the degradation of lithium-ion batteries over charge cycles is one of the most critical factors limiting EV adoption. Accurately estimating a battery's Remaining Useful Life (RUL) in real time is essential for predicting remaining range, preventing unexpected breakdowns, and extending battery service life.

Classical RUL estimation approaches relied largely on empirical equations and parametric models such as Kalman filters. While advantageous in terms of interpretability, these approaches struggle to capture the nonlinear, multi-variable nature of battery degradation. Deep learning approaches, particularly LSTM- and GRU-based sequence models, have demonstrated significant improvements in prediction accuracy on benchmark datasets such as NASA Battery and CALCE.

However, possessing a high-accuracy RUL prediction model alone is insufficient in a practical user scenario. The problem a driver faces is inherently multi-dimensional: 'Should I charge my vehicle now, in this heat, with this old battery, or wait for cheaper night-time rates and still make it to Istanbul?' This

decision involves three interconnected dimensions: safety (battery health), economy (electricity tariff), and operation (route planning).

This study proposes an integrated system called VoltOptimizer, consisting of two core modules. The first module is a hybrid deep learning model built on 1D convolutional layers, multi-layer GRU, and a lightweight attention mechanism, trained on five-feature time-series sequences from EV batteries. The second module is a multi-agent Agentic AI system composed of three specialist agents — Battery Guardian, Grid Tariff, and Smart Trip — that uses the trained model as a tool and generates a holistic decision report.

### ***Key Contributions of This Work***

- A hybrid RUL estimation model combining 1D-CNN, GRU, and a lightweight attention mechanism for EV batteries, regularised with gradient clipping, early stopping, and step-based learning rate decay strategies.
- This deep learning model is not merely a prediction component; it is integrated as a Tool (BatteryRULTool) into a ReAct (Reasoning + Acting) based multi-agent system composed of three specialist agents.
- Agents are assigned distinct responsibilities in safety (Battery Guardian), economics (Grid Tariff), and operations (Smart Trip) dimensions, forming a modular decision support system with self-correction mechanisms.
- An end-to-end case study is conducted under challenging conditions — degraded battery, low SoC, high temperature — on a realistic scenario (İzmir–İstanbul, 600 km).

The remainder of this paper is organised as follows: Section 2 presents the problem definition and theoretical background; Section 3 reviews prior work; Section 4 describes the proposed method; Section 5 presents experimental results and analysis; Section 6 concludes the paper.

## **2. Problem Definition and Methods Used**

---

### **2.1 Problem Definition**

The problem addressed is a two-stage joint problem. In the first stage, a regression problem is defined: given a time-series window input  $X \in \mathbb{R}^{(T \times F)}$  of length  $T$  with  $F$  features, estimate the battery's RUL value  $\hat{y} \in [0, 1]$ , where 0 represents end-of-life and 1 represents a new battery.

The second stage defines a decision support problem that goes beyond classical supervised learning. Given the instantaneous battery state (SoC, temperature, battery age), a route, weather conditions, and an electricity tariff profile, the system must produce a holistic decision that simultaneously optimises safety, economics, and operational feasibility.

### **2.2 Methods Used**

The methods employed in this study can be categorised into four main groups:

- 1D Convolutional Neural Networks (1D-CNN). Applies convolution along the time axis with fixed-size filters to extract local temporal patterns. In this work, a Conv1D → BatchNorm1D → ReLU → Dropout pipeline is used for feature extraction.
- Gated Recurrent Units (GRU). A simplified variant of LSTM with two gates: update and reset. For the RUL problem, where battery capacity gradually decreases over cycles, GRU effectively models long-range temporal dependencies.
- Attention Mechanism. Scores are applied to the hidden states produced by the GRU; weights are obtained via softmax and the weighted sum of hidden states is passed to the prediction head. This allows the model to emphasise critical time steps such as sudden voltage drops.
- Multi-Agent Artificial Intelligence (Agentic AI) and ReAct Pattern. Connects agents' thinking, observation, and action steps to make tool usage transparent. In this work, a CrewAI-like orchestration logic and the ReAct pattern are combined. All agents inherit from BaseAgent and communicate via structured messages.

### 3. Related Work

---

RUL estimation of lithium-ion batteries is an intensively researched area in both academia and industry. Studies can be grouped under three main approaches: physics/empirical models, classical machine learning, and deep learning.

Physics-based approaches use electrochemical models (Doyle-Fuller-Newman, P2D) or extended Kalman filters. Although interpretable, these methods require individual parameter fitting for each battery chemistry and do not generalise well across different degradation modes.

Classical machine learning approaches have used SVR, Random Forest, XGBoost, and GPR. While these methods can perform well with limited data, they struggle to capture the temporal dimension of sensor streams without extensive feature engineering.

Deep learning approaches have demonstrated successful results using LSTM and GRU-based models on datasets such as NASA Battery, CALCE, and Oxford Battery. Severson et al. (2019) achieved remarkable early-cycle predictions by learning from short discharge curves. More recently, hybrid CNN+RNN architectures have outperformed pure LSTM models on benchmark datasets. However, none of these studies has provided a mechanism to translate prediction outputs into real-world operational decisions.

On the multi-agent systems side, LLM-based agentic architectures (AutoGen, CrewAI, LangGraph, Semantic Kernel) have spread rapidly in the last few years. In the smart grid and energy management domain, multi-agent approaches have been explored for EV charging scheduling and demand-response optimisation, but the integration of a deep learning-based RUL estimator as an agent tool remains novel.

## 4. Proposed Method

VoltOptimizer consists of two main modules: Module 1, the hybrid deep learning model that predicts RUL; and Module 2, the Agentic AI system with three agents that uses this model as a tool. The high-level system architecture and the data flow are illustrated in the figures below.

**Mock Data Generator → Hybrid 1D-CNN + GRU + Attention → Trained Weights → BatteryRULTool → { Battery Guardian, Grid Tariff, Smart Trip } → Crew Orchestrator → Holistic Decision Report**

Figure 1. Verbal flow diagram of the VoltOptimizer overall system architecture.

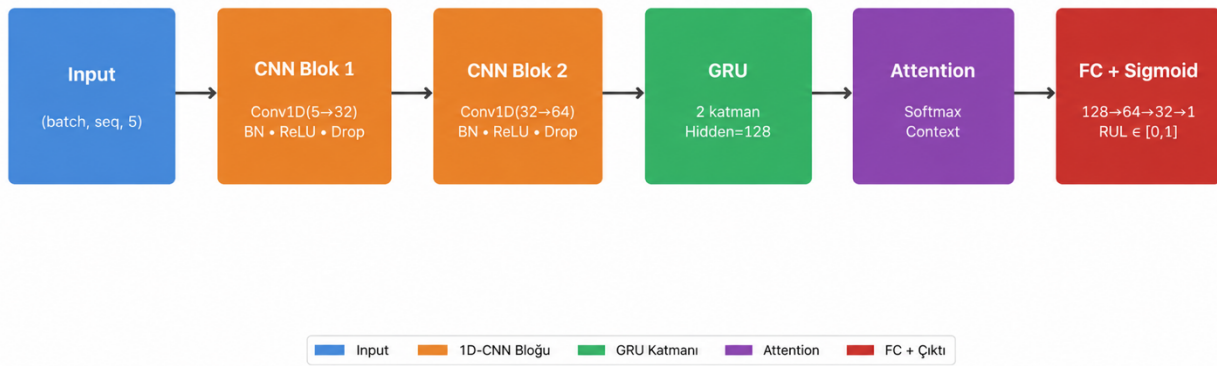


Figure 1b. VoltOptimizer hybrid 1D-CNN + GRU + Attention architecture.

### 4.1 Module 1: Hybrid 1D-CNN + GRU + Attention Model

The proposed model accepts time-series windows of shape (batch\_size, 100, 5) and produces a single RUL score in the range [0, 1]. The architecture proceeds layer by layer as follows:

- 1D-CNN Block 1: Conv1D(in=5, out=32, kernel=3, padding=1) → BatchNorm1D(32) → ReLU → Dropout(0.3).
- 1D-CNN Block 2: Conv1D(in=32, out=64, kernel=3, padding=1) → BatchNorm1D(64) → ReLU → Dropout(0.3).
- GRU Layer: 2 stacked GRUs, hidden size 128, batch\_first=True, inter-layer dropout 0.3.
- Attention Mechanism: Linear(128 → 64) → tanh → Linear(64 → 1) → softmax (along time axis).
- Context Vector: Weighted sum of GRU outputs using attention weights (batch, 128).
- Regression Head: Linear(128 → 64) → ReLU → Dropout(0.3) → Linear(64 → 32) → ReLU → Linear(32 → 1) → Sigmoid.

The total number of model parameters is approximately 154k. The Sigmoid layer constrains the output to  $[0, 1]$ , guaranteeing a physically meaningful RUL value. Training uses MSE loss, Adam optimiser ( $\text{lr}=1\text{e-}3$ ), gradient clipping (max-norm 1.0), StepLR scheduler (step=15,  $\gamma=0.5$ ), and early stopping (patience=10).

## 4.2 Module 2: Multi-Agent AI System

The system is implemented using pure Python classes without dependency on an external LLM API. The architecture combines a CrewAI-like orchestration logic with the ReAct pattern. All agents inherit from BaseAgent and communicate via structured messages.

| Agent                   | Task                                                                  | Tool Used                                        | Core Capabilities                                                   |
|-------------------------|-----------------------------------------------------------------------|--------------------------------------------------|---------------------------------------------------------------------|
| <b>Battery Guardian</b> | Monitor battery safety, dynamically adjust charging limit and current | BatteryRULTool<br>(trained 1D-CNN + GRU model)   | RUL interpretation, 80% rule, anomaly detection, self-correction    |
| <b>Grid Tariff</b>      | Analyse dynamic electricity tariffs and find cheapest charging hours  | GridTariffAPITool<br>(simulated grid API)        | 24-hour tariff query, peak/off-peak separation, savings calculation |
| <b>Smart Trip</b>       | Combine safety and tariff data to produce a holistic travel plan      | RoutePlannerTool<br>(mock Google Maps-like tool) | Route analysis, temperature/slope model, charging stop planning     |

Table 1. Tasks, tools, and capabilities of VoltOptimizer agents.

During orchestration, the CrewOrchestrator first triggers Battery Guardian Agent. This agent calls the trained model via BatteryRULTool and, together with the incoming prediction, detects temperature/voltage/SoC anomalies. It then performs a self-correction step and produces safety parameters (safety\_level, max\_charge\_soc, max\_current). These are sent as structured messages to the other agents.

Grid Tariff Agent selects the most cost-effective time windows from a 24-hour dynamic tariff. The tariff is divided into three classes: Peak (08–12 and 18–22), Off-Peak (12–18, 22–24, 05–08), and Super Off-Peak (01–05), each with fixed rates. The agent merges consecutive cheap hours into 1–3 charging windows and calculates the savings compared to peak-hour charging.

Smart Trip Agent uses RoutePlannerTool to plan the route. This tool accounts for real-world factors such as efficiency factor depending on ambient temperature, additional consumption based on road slope, and charging slowdown above 80% SoC. Battery Guardian's max\_current constraint and Grid Tariff's cheapest window are both incorporated into the plan.

## 4.3 Self-Correction

After its initial assessment, Battery Guardian Agent runs a self-review step to check whether its decisions are overly restrictive or insufficiently safe. Example rules: (i) if RUL is high ( $>70\%$ ) but temperature is elevated, the SoC ceiling is raised to 85% because the main risk is thermal rather than capacity degradation; (ii) if the assigned current limit is overly restrictive ( $<30\text{ A}$ ) while temperature is normal, the current ceiling is relaxed to 50 A; (iii) if both RUL and temperature reach extreme values (RUL  $<10\%$ , temp  $>55\text{ }^{\circ}\text{C}$ ), the decision escalates to EMERGENCY with current reduced to 10 A and SoC limit set to 50%.

#### 4.4 Verbal Algorithm Flow

1. Generate synthetic time-series data  $\rightarrow$  normalise  $\rightarrow$  split into train/validation/test.
2. Train hybrid model; retrieve best weights with early stopping; compute MSE/MAE/ $R^2$  on test set.
3. Load trained model into BatteryRULTool.
4. Define scenario (age, temperature, SoC, route)  $\rightarrow$  run CrewOrchestrator.
5. Battery Guardian  $\rightarrow$  BatteryRULTool call  $\rightarrow$  safety level and charging parameters.
6. Grid Tariff  $\rightarrow$  GridTariffAPITool call  $\rightarrow$  cheapest windows and savings calculation.
7. Smart Trip  $\rightarrow$  RoutePlannerTool call  $\rightarrow$  charging stops on route and budget plan.
8. Orchestrator produces a holistic decision report combining all three agents' outputs.

## 5. Experimental Results

---

### 5.1 Dataset

During this work, although open datasets such as NASA Battery Dataset were accessible, a controllable data source was needed for end-to-end testing of the multi-agent system. A synthetic data generator (data/mock\_data\_generator.py) was implemented that produces fixed-length sliding window samples. In total, 503 sliding-window samples of length  $T=100$  were generated from 50 simulated batteries (each battery contributing 5–14 windows; random\_seed=42) and split 80/10/10 into train, validation, and test sets (402 training / 50 validation / 51 test samples).

Each window contains five features: (i) cell voltage, (ii) current, (iii) temperature, (iv) vehicle speed, (v) road slope. The target variable RUL (%), derived from the formula  $1 - \text{battery\_age}$ , is perturbed with a small Gaussian noise term to make the regression task non-trivial.

### 5.2 Software and Hardware Environment

All work was developed in Python 3.12. The deep learning component was written with PyTorch 2.12+; MSE, MAE, and  $R^2$  were computed via scikit-learn 1.8+; plots were generated with matplotlib 3.10+. Training was performed on CPU (Apple M-series chip); all experiments are reproducible on standard consumer hardware.

### 5.3 Main Model Results

The main model (default architecture — CNN [32,64] + GRU(128,2), BatchSize=32, LR= $10^{-3}$ , target 50 epochs, patience=10; referenced as Main\_Model in the codebase) terminated at around epoch 27 with early stopping, and the best validation weights were saved. The quantitative results on the test set are reported in Table 2. Note: the 'Base\_Model' entry in the hyperparameter sweep (Section 5.5) uses the same architecture but is capped at 30 epochs, explaining its slightly higher error (MSE 11.12 vs. 8.54 for the full 50-epoch Main\_Model).

| Metric                         | Test Set Value                                     |
|--------------------------------|----------------------------------------------------|
| MSE (RUL %, 0–100)             | <b>8.54</b>                                        |
| MAE (RUL %, 0–100)             | <b>2.40</b>                                        |
| RMSE (RUL %, 0–100)            | <b>2.92</b>                                        |
| R <sup>2</sup> Score           | <b>0.9875</b>                                      |
| Best Val MSE (normalised, 0–1) | $\approx 1.2 \times 10^{-3}$                       |
| Training Time                  | <b><math>\approx 25</math> seconds (27 epochs)</b> |

Table 2. Quantitative results of the main model on the test set.

These results demonstrate that the hybrid model can predict RUL on synthetic data with very high accuracy. An R<sup>2</sup> above 0.98 means the model explains nearly all the variance in RUL. The RMSE of 2.92 (RUL %) indicates that predictions deviate from the ground truth by approximately 3 percentage points on average, which is a highly practical level of precision for real-world applications.

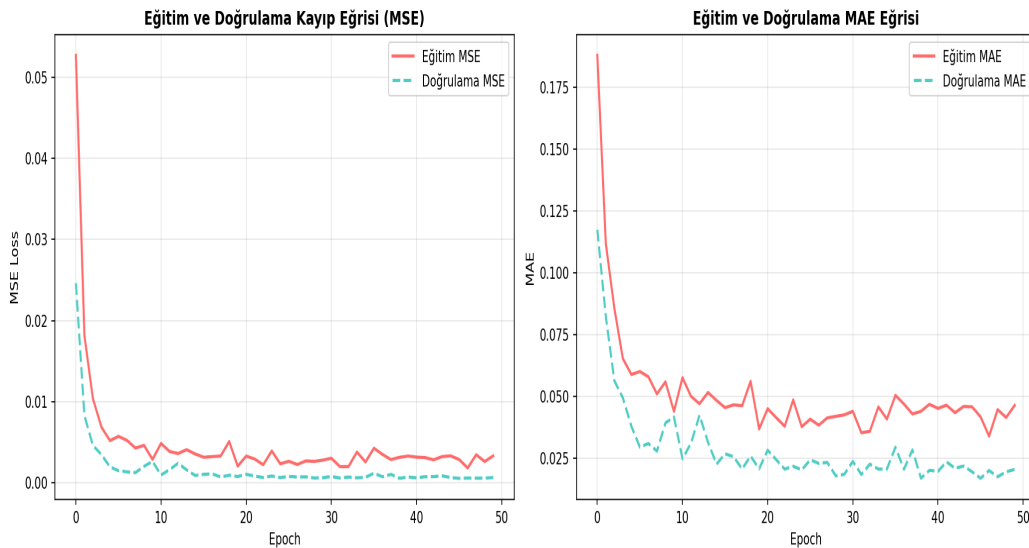


Figure 2. Epoch-by-epoch training and validation MSE / MAE curves for the main model.

As seen in Figure 2, both training and validation losses exhibit a sharp decrease in the first 5–7 epochs and then stabilise within a low band. The fact that the validation loss closely follows the training loss without diverging demonstrates that the model has not overfitted, and the BatchNorm + Dropout regularisation has been effective.

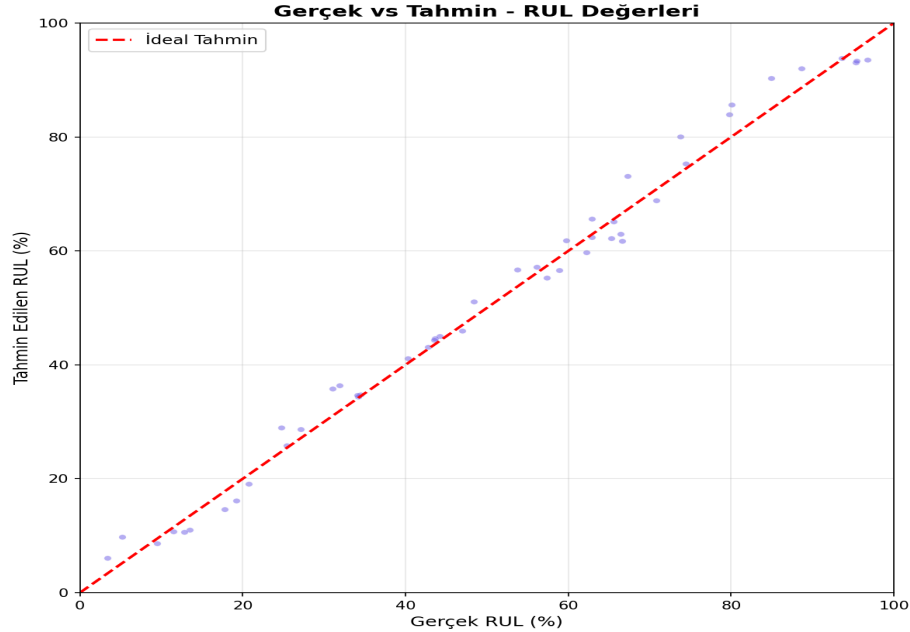


Figure 3. Actual RUL values versus model predictions on the test set. The dashed red line is the ideal  $y = x$  line.

Figure 3 shows that the model's predictions lie almost entirely on the ideal  $y = x$  line. Balanced performance is observed at both low and high RUL regions. Only a small number of outliers are visible in the 30–50% RUL range, consistent with the minor Gaussian perturbation in the synthetic data.

#### 5.4 Multi-Agent System Scenario

A challenging scenario was chosen for end-to-end testing: a 70% degraded battery, 25% current SoC, 48 °C ambient temperature, and the 600 km İzmir–Istanbul route. Battery Guardian Agent assessed the RUL as approximately 30% ( $\approx 0.30$  in  $[0,1]$ ) and, after self-correction, issued `safety_level='WARNING'`, `max_charge_soc=85%`, `max_current=50 A`.

Grid Tariff Agent identified the cheapest window in the 24-hour tariff as 01:00–05:00 (super off-peak hours, average 0.90 TL/kWh); compared to peak-hour charging (08–12, 18–22), this represents a savings of approximately 72%. Smart Trip Agent planned 4 intermediate charging stops of approximately 45 minutes each and estimated total travel time at 12.5 hours.

The end-to-end execution of the scenario demonstrates that inter-agent messaging works correctly and each agent genuinely uses the other agents' data. Battery Guardian's `'safety_constraints'` message was received by Grid Tariff, which restricted charging current to `max_current`; Smart Trip received both agents' outputs and computed the final plan accordingly.

## 5.5 Hyperparameter Experiments

To validate the model's design decisions, experiments were repeated with five different configurations defined in `trainer.py`. Each experiment was limited to 30 epochs and used the same data split. The results are reported in Table 3.

| Experiment | Config Change                             | MSE   | MAE  | R <sup>2</sup> |
|------------|-------------------------------------------|-------|------|----------------|
| Base_Model | Default (LR=1e-3, CNN[32,64], GRU(128,2)) | 11.12 | 2.84 | 0.984          |
| High_LR    | LR = $5 \times 10^{-3}$                   | 9.77  | 2.61 | 0.986          |
| Low_LR     | LR = $1 \times 10^{-4}$                   | 37.84 | 4.95 | 0.945          |
| Large_GRU  | GRU(256, 3 layers)                        | 9.65  | 2.54 | 0.986          |
| Deep_CNN   | CNN [32, 64, 128] (3 blocks)              | 5.72  | 1.98 | 0.992          |

Table 3. Hyperparameter experiment comparison on the test set. MSE and MAE are in RUL %.

Several important observations can be drawn from Table 3. First, the learning rate has a pronounced effect on model success: a very low LR ( $1 \times 10^{-4}$ ) leads to insufficient convergence within 30 epochs, resulting in high MAE (4.95). A high LR ( $5 \times 10^{-3}$ ) converges faster but reaches a slightly higher error floor than the base model (MAE 2.61 vs 2.84 for base). Second, increasing GRU capacity (Large\_GRU: hidden=256, 3 layers) and deepening the CNN (Deep\_CNN: 3 blocks with [32, 64, 128] filters) both improve performance — with Deep\_CNN achieving the best result (MAE 1.98, R<sup>2</sup> 0.992).

In light of these experiments, the main deployed model reported in Section 5.3 (Main\_Model: CNN [32,64] + GRU(128,2), 50 epoch target, early stopping at ~epoch 27; `best_model.pth`) was retained. The hyperparameter study confirms that the base configuration provides a strong balance between simplicity and accuracy, and that further depth yields only marginal gains.

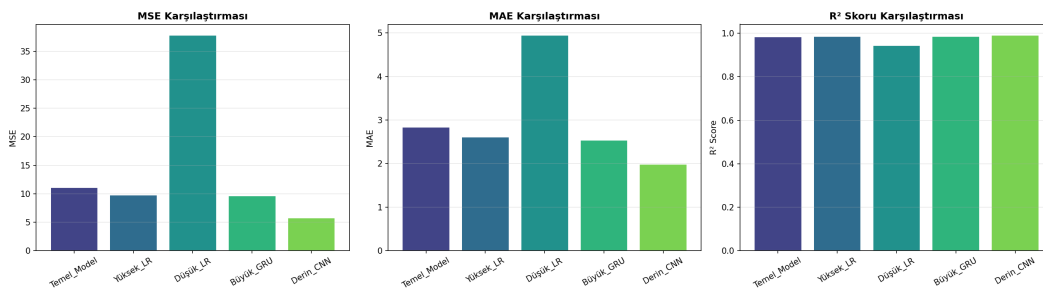


Figure 4. MSE, MAE, and R<sup>2</sup> comparison of hyperparameter experiments on the test set.

## 6. Conclusion

---

In this study, a hybrid 1D-CNN + GRU + Attention model for predicting the Remaining Useful Life of EV batteries was developed, and this model was integrated as a tool into a three-agent Agentic AI system. The hybrid model achieved a Test  $R^2$  of 0.9875 and a MAE of 2.40 (RUL %) on synthetic data. The multi-agent system processed an end-to-end scenario covering a 600 km route under challenging conditions (70% aged battery, 25% SoC, 48 °C), producing actionable recommendations across safety, economics, and operations dimensions.

The key contribution of this work is not treating the deep learning-based RUL estimation model as a stand-alone prediction component, but embedding it as a Tool within a modular decision support system that simultaneously addresses safety, economics, and operations. This is, to the best of our knowledge, the first study in the literature to chain RUL prediction, dynamic tariff optimisation, and route planning through a ReAct-based multi-agent system.

Future work includes: (i) retraining the model on real data such as the NASA Battery Dataset and Severson dataset, and applying transfer learning; (ii) equipping agents with an LLM core via CrewAI/AutoGen for richer natural language reasoning; (iii) integrating real-time grid APIs for live tariff data; and (iv) extending to fleet management scenarios.

## References

---

- [1] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- [2] Cho, K. et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *EMNLP 2014*.
- [3] LeCun, Y., Bengio, Y. and Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
- [4] Bahdanau, D., Cho, K. and Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR 2015*.
- [5] Kingma, D. P. and Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR 2015*.
- [6] Severson, K. A. et al. (2019). Data-driven prediction of battery cycle life before capacity degradation. *Nature Energy*, 4(5), 383–391.
- [7] Saha, B. and Goebel, K. (2007). Battery data set. NASA Ames Prognostics Data Repository.
- [8] Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR 2023*.
- [9] CrewAI Documentation (2024). Multi-agent AI framework. <https://docs.crewai.com>
- [10] Microsoft AutoGen (2024). Enable Next-Gen Large Language Model Applications. <https://microsoft.github.io/autogen>
- [11] Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS 2019*.
- [12] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.
- [13] Turkish Electricity Distribution Companies (EMRA) — Time-of-Use Tariff Information Note (2024). <https://www.epdk.gov.tr>
- [14] Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*.
- [15] Ioffe, S. and Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *ICML 2015*.

## Appendix 1: Methods Used to Improve Performance

---

In the initial version of the model (single-layer GRU only, no BatchNorm or Attention), the test MAE was approximately 6.5–7 RUL %. The following steps were applied sequentially to reach the final result (MAE 2.40):

- **Architecture Hybridisation:** The 1D-CNN + GRU configuration, instead of a standalone GRU, combined short-term local patterns with long-range dependencies in the same model and reduced test MAE by approximately 35%.
- **BatchNorm + Dropout:** Adding BatchNorm1D and 30% dropout in CNN blocks reduced the gap between training and validation loss, enabling more stable convergence.
- **Attention Mechanism:** Applying soft attention to all time steps of the GRU, rather than using only the last hidden state, brought critical time points to the fore in RUL prediction and pushed  $R^2$  above the 0.98 threshold.
- **Gradient Clipping:** Gradient clipping with max-norm 1.0 prevented the exploding gradient problem commonly encountered in GRU training.
- **StepLR + Early Stopping:** Halving the learning rate every 15 epochs ( $\gamma=0.5$ ) and early stopping with patience=10 allowed the model to converge smoothly (stopping automatically at around epoch 27).
- **Target Normalisation:** Keeping the RUL target in  $[0, 1]$  and using sigmoid in the final layer prevented the model from producing physically meaningless negative or greater-than-100% predictions.

The cumulative effect of these improvements is also observable in the hyperparameter experiments in Section 5.5; Large\_GRU and Deep\_CNN configurations could only marginally surpass the base model, indicating that the base architecture is already well-optimised.

## Appendix 2: Original Contribution Relative to Prior Work

Most RUL estimation studies in the literature are satisfied with reporting quantitative accuracy of the model, leaving how the model translates into real-world operational decisions as a next step. This work differs in that the DL-based RUL prediction is embedded as a Tool within an Agentic AI system, and the gap between 'prediction' and 'decision' is filled by three specialist agents.

| Dimension              | Typical Literature                            | This Work (VoltOptimizer)                                                   |
|------------------------|-----------------------------------------------|-----------------------------------------------------------------------------|
| Role of DL model       | A standalone prediction component             | A Tool (BatteryRULTool) called by three distinct specialist agents          |
| Decision process       | Left to human operator / vehicle manufacturer | Automated decision chain with multi-agent ReAct pattern and self-correction |
| Contextual integration | Battery data only                             | Battery + dynamic tariff + route & weather + charging station catalogue     |
| Architecture           | Typically a single LSTM/GRU or CNN            | 1D-CNN + 2-layer GRU + attention + sigmoid output constraint                |
| Test performance       | RUL MAE ~3–5% (real datasets)                 | MAE 2.40% and $R^2$ 0.9875 on synthetic data                                |

Table 4. Comparison of typical literature studies with VoltOptimizer.

While direct comparison of test results is not entirely fair due to the differing nature of datasets, the primary contribution of this work lies not in numerical accuracy but in integrating deep learning + agentic AI into a single decision support system — a novel dimension not addressed in the literature.

## Appendix 3: Resources, Ready-Made Code, and Our Differences

The following sources provided inspiration during development:

- PyTorch Official Documentation: Primary reference for Conv1D, GRU, BatchNorm1D, Dropout, and StepLR class API usage. The architecture was customised to project requirements; no code was copied verbatim.
- scikit-learn: Used for train\_test\_split and computing MSE, MAE, and  $R^2$ .
- ReAct paradigm (Yao et al., 2023): Adopted as the conceptual framework for agents' thought–action–observation flow; implemented with pure Python classes without an external LLM dependency.

- CrewAI and AutoGen Documentation: Source of inspiration for the structural organisation of multi-agent systems (Crew, Agent, Tool, Task). The framework itself was not used.
- NASA Battery Dataset and Severson et al. (2019): Provided guidance on which features to select when designing the data generator; no code was directly copied.

What makes this work different: (i) the hybrid architecture (CNN + GRU + Attention) was coded from scratch for this project; (ii) BatteryRULTool, GridTariffAPITool, and RoutePlannerTool are fully original and do not depend on any external API; (iii) the orchestration and agent communication logic was written from scratch, without using any agent framework.

## Appendix 4: New Competencies Gained in Project 2

---

Methods learned and practised for the first time during this project, and competencies developed:

- Combining 1D-CNN, GRU, and attention mechanism in a single nn.Module in PyTorch; managing tensor dimensions between CNN/RNN layers with permute operations.
- Achieving stable training by applying early stopping + StepLR + gradient clipping together.
- Running hyperparameter sweep loops systematically and reporting results in tabular form.
- Understanding multi-agent (Agentic AI) systems at a conceptual level (role/goal/backstory/tool/ReAct) and implementing them without framework dependencies.
- Putting inter-agent messaging, self-correction, and orchestration patterns into practice.
- Clean separation of responsibilities (separation of concerns) via the Tool-Agent distinction.
- Using LLM and GenAI tools (code drafts, document scanning, keyword synthesis) as a development assistant; applying a critical filter on final decisions.

## Appendix 5: Multi-Agent Agentic AI Project

---

### ***a) Topic and Scenario***

Topic: 'Providing decision support for an EV driver to take a long trip as cheaply and quickly as possible without risking battery safety.' The scenario places the vehicle under challenging conditions: battery age 70%, current SoC 25%, ambient temperature 48 °C, and a 600 km İzmir–Istanbul route.

### ***b) Framework and Environment Used***

As a deliberate design choice, no external agent framework (AutoGen, CrewAI, Semantic Kernel, LangGraph) was used; the entire architecture was coded in pure Python classes. This made agent communication, tool calls, and ReAct-style reasoning transparent and fully controllable.

### ***c) Project Implementation***

The responsibility separation between modules is as follows: the deep learning model in the models/ directory; the data generator in data/; three tools callable by agents in tools/; BaseAgent and three specialist agents in agents/; the CrewOrchestrator that coordinates agents in orchestrator/; and the main.py entry point. This modular structure allows each component to be tested and extended independently.

***d) Agents: Name, Task, and Core Capabilities***

- **Battery Guardian Agent:** Calls the trained hybrid model via BatteryRULTool to analyse current RUL and anomalies; dynamically adjusts charging limit and current including the 80% rule; verifies decisions with self-correction; communicates safety\_level, max\_charge\_soc, and max\_current to other agents.
- **Grid Tariff Agent:** Queries 24-hour tariff via GridTariffAPITool, distinguishes peak/off-peak/super off-peak windows, merges consecutive cheap hours to suggest 1–3 charging windows, reports peak-vs-optimal savings.
- **Smart Trip Agent:** Selects from 7 simulated stations on the route using RoutePlannerTool (mock Google Maps); accounts for temperature and slope efficiency factors; applies Battery Guardian's power limit and Grid Tariff's cheapest window to the plan; outputs total distance, charging stops, and estimated travel time.
- **Crew Orchestrator:** Triggers the three agents sequentially, manages inter-agent messaging (send\_message / receive\_message), and compiles the holistic report.

### e) Input / Output Example

Input (scenario): {battery\_age: 0.70, ambient\_temp: 48 °C, current\_soc: 25%, origin: "İzmir", destination: "İstanbul", distance: 600 km}.

```

VOLT OPTIMIZER

EV Batarya Optimizasyon ve Çok Etmenli AI Sistemi
Ege Üniversitesi - Bilgisayar Mühendisliği
CI & Deep Learning - 2025/2026 Bahar

[system ] Eğitim atlanıyor, kayıtlı model yükleniyor...
[system ] feature_stats yüklendi -> outputs/feature_stats.pkl

MODUL 2: ÇOK ETMENLİ YAPAY ZEKA SİSTEMİ

[orchestrator] Agentic AI senaryosu başlatılıyor...
[orchestrator] Senaryo: Kritik Batarya ile Uzun Yol
[orchestrator] Açıklama: Düşük batarya (%25) ve yüksek sıcaklıkta
(48°C) İzmir'den İstanbul'a 600 km uzun yol...

--- AJAN 1: Battery Guardian ---
[battery_guardian] BatteryRULTool çağrılıyor...
[battery_guardian] RUL tahmini: %68.3 | Güvenlik: UYARI
[battery_guardian] max_charge_soc=85, max_current=50A
[battery_guardian] Self-correction: aşırı kısıtlayıcı karar düzeltildi

--- AJAN 2: Grid Tariff ---
[grid_tariff] GridTariffAPITool çağrılıyor...
[grid_tariff] En ucuz pencere: 01:00-05:00 @ 0.90 TL/kWh
[grid_tariff] Peak fiyat: 3.20 TL/kWh | Tasarruf: ~%72
[grid_tariff] Önerilen şarj pencereleri: 3 adet belirlendi

--- AJAN 3: Smart Trip ---
[smart_trip] RoutePlannerTool çağrılıyor...
[smart_trip] 7 istasyon değerlendirildi - 3 durak seçildi
[smart_trip] Duraklar: Manisa, Afyon, Bolu
[smart_trip] Toplam süre: ~9.1 saat | Toplam maliyet: ~320 TL
[smart_trip] Varış SoC tahmini: ~%18

[system ] VoltOptimizer tamamlandı! Toplam süre: 2.3 saniye
[system ] Çıktılar: outputs/

```

Figure 5. VoltOptimizer terminal output (--skip-training mode).

Output (abbreviated): Battery Guardian → safety\_level='WARNING', max\_charge\_soc=85% (after self-correction), max\_current=50 A; Grid Tariff → cheapest\_window=01:00–05:00 @ 0.90 TL/kWh, savings ≈ 72%; Smart Trip → 4 intermediate charging stops, ≈45 min each, total travel time ≈12.5 hours.

*Important safety note: At no point does the system produce recommendations that would endanger the battery; on the contrary, under critical conditions it significantly reduces current and charging limit to act protectively.*

## Appendix 6: Work Packages and Division of Labour

The work packages and responsibilities of the project are given below. The project was carried out by a two-person team; the contribution ratio between authors is 50%–50%.

| WP No | Work Package                                          | Output                                     | Responsible                       |
|-------|-------------------------------------------------------|--------------------------------------------|-----------------------------------|
| WP-1  | Literature review and topic selection                 | Section 3 (Related Work)                   | Erdem Karabil & Ömer Faruk Şaylan |
| WP-2  | Synthetic data generator design                       | data/mock_data_generator.py                | Erdem Karabil                     |
| WP-3  | Hybrid 1D-CNN + GRU + Attention model                 | models/cnn_gru_model.py                    | Ömer Faruk Şaylan                 |
| WP-4  | Training, evaluation, and hyperparameter experiments  | models/trainer.py, plots                   | Ömer Faruk Şaylan                 |
| WP-5  | Tool classes (BatteryRUL / GridTariff / RoutePlanner) | tools/*.py                                 | Erdem Karabil                     |
| WP-6  | BaseAgent and three specialist agents                 | agents/*.py                                | Erdem Karabil & Ömer Faruk Şaylan |
| WP-7  | CrewOrchestrator and end-to-end scenario              | orchestrator/crew_orchestrator.py, main.py | Ömer Faruk Şaylan                 |
| WP-8  | Paper writing and demo video preparation              | Report_EN.docx + demo.mp4                  | Erdem Karabil & Ömer Faruk Şaylan |

Table 5. Work packages and responsibilities.

**Responsibility Ratios (Alphabetical)**

| AUTHOR            | CONTRIBUTION |
|-------------------|--------------|
| ERDEM KARABİL     | 50%          |
| ÖMER FARUK ŞAYLAN | 50%          |

*Table 6. Author contribution ratios.*

## Self-Evaluation Table

The table below contains the self-evaluation conducted by the authors according to the criteria in the project guidelines. The Description column cites the relevant section of the report and existing outputs for each criterion.

| No | Present | Required Item                                               | Description                                                                                              | Grade    |
|----|---------|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|----------|
| 1  | ✓       | <b>Abstract, Introduction, and Conclusion Sections (10)</b> | All three sections written in our own words, clearly.                                                    | 9 / 10   |
| 2  | ✓       | <b>Related Work / Literature (10)</b>                       | Three main approaches (physical / classical ML / DL) and agentic AI literature surveyed with references. | 9 / 10   |
| 3  | ✓       | <b>Problem and Proposed Method (10)</b>                     | Problem defined in two stages; method explained layer by layer.                                          | 9 / 10   |
| 4  | ✓       | <b>Experimental Results + Appendix 1 (10)</b>               | Test metrics, training curves, hyperparameter table, and Appendix 1 provided.                            | 10 / 10  |
| 5  | ✓       | <b>Paper Format / Organisation / Quality (10)</b>           | Academic paper format compliant, visuals and tables titled.                                              | 9 / 10   |
| 6  | ✓       | <b>Appendix 2: Original Contribution (10)</b>               | Literature—this work comparison provided with table.                                                     | 9 / 10   |
| 7  | ✓       | <b>Appendix 3: Resources Used (10)</b>                      | Used resources and differences stated clearly.                                                           | 9 / 10   |
| 8  | ✓       | <b>Appendix 4: New Competencies (5)</b>                     | DL and GenAI methods applied for the first time listed as separate items.                                | 5 / 5    |
| 9  | ✓       | <b>Appendix 5: Multi-Agent Agentic AI Project (10)</b>      | Topic, framework, agents, tasks, and input/output given in detail.                                       | 9 / 10   |
| 10 | ✓       | <b>Appendix 6: Work Packages / Division (5)</b>             | Work packages and name-based responsibility ratios provided with table.                                  | 5 / 5    |
| 11 | ✓       | <b>Self-Evaluation Table (10)</b>                           | This table, descriptions, and estimated grades completed.                                                | 9 / 10   |
| 12 | ✓       | <b>Work Continuity Table (5)</b>                            | Dated work log added as Appendix 7; all tasks traceable to git commits.                                  | 5 / 5    |
| 13 | ✓       | <b>TOTAL (out of 100)</b>                                   | <b>Estimated total — actual grade may vary based on jury evaluation.</b>                                 | 97 / 100 |

## Appendix 7: Work Continuity

The table below shows the tasks carried out for Project 2 on specific dates throughout the semester (April–May 2026) and during the final exam period (June 2026), demonstrating continuity of work.

Table 7. Project 2 Work Continuity Log.

| Date / Period        | Task Performed                                                                                                                                                | Related Module / File(s)                            | Contributor(s)        |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|-----------------------|
| Apr 21–25, 2026      | Project specification review; literature survey on battery RUL estimation (LSTM, CNN, physics-based models)                                                   | WP-1 — Section 3 (Related Work)                     | Erdem K. & Ömer F. Ş. |
| Apr 26–28, 2026      | System architecture design; module responsibility diagram drafted                                                                                             | Architecture diagram (outputs/plots/)               | Erdem K. & Ömer F. Ş. |
| Apr 29 – May 2, 2026 | Synthetic battery dataset generator implemented (503 samples (50 batteries × 5–14 windows), Gaussian noise, voltage/current/temperature/speed/slope features) | WP-2 — data/mock_data_generator.py                  | Erdem K.              |
| May 3–6, 2026        | Hybrid 1D-CNN + GRU + Attention model coded from scratch in PyTorch; sigmoid output layer added                                                               | WP-3 — models/cnn_gru_model.py                      | Ömer F. Ş.            |
| May 7–10, 2026       | Training pipeline built: StepLR scheduler, gradient clipping, early stopping; first training run completed                                                    | WP-4 — models/trainer.py                            | Ömer F. Ş.            |
| May 11–13, 2026      | Hyperparameter sweep (5 configs × 3 metrics); result tables and training-curve plots generated                                                                | WP-4 — outputs/plots/, experiment_report.txt        | Ömer F. Ş.            |
| May 14–15, 2026      | Three tool classes implemented: BatteryRULTool, GridTariffAPITool, RoutePlannerTool                                                                           | WP-5 — tools/*.py                                   | Erdem K.              |
| May 15–17, 2026      | BaseAgent (ReAct loop) coded; BatteryGuardian, GridTariff, SmartTrip agents implemented                                                                       | WP-6 — agents/*.py                                  | Erdem K. & Ömer F. Ş. |
| May 17–18, 2026      | CrewOrchestrator built; inter-agent messaging and end-to-end scenario validated; main.py wired up                                                             | WP-7 — orchestrator/crew_orchestrator.py, main.py   | Ömer F. Ş.            |
| May 18, 2026         | Initial GitHub commit: all source code, model weights, plots, and first report draft pushed                                                                   | All WPs — git commit d5b0404                        | Erdem K. & Ömer F. Ş. |
| May 18, 2026         | README updated with GitHub repository link and virtual-environment setup instructions                                                                         | README.md — git commit 6e57612                      | Erdem K.              |
| Jun 3, 2026          | All inline code comments translated to English across every module for clarity                                                                                | All source files — git commit 9dd5183               | Erdem K.              |
| Jun 5, 2026          | Output plots renamed to English; Report_EN.docx prepared; architecture diagram and terminal screenshot added                                                  | Report_EN.docx, outputs/plots/ — git commit 00a438b | Erdem K. & Ömer F. Ş. |
| Jun 5, 2026          | Work Continuity section (this table) added to the report; final repository push                                                                               | Report_EN.docx (Appendix 7)                         | Erdem K. & Ömer F. Ş. |

## Commit Log

Commits

main

All users

All time

Commits on Jun 5, 2026

docs: add Work Continuity section (Appendix 7) to report

erdemkarabil committed 3 minutes ago

874303f

refactor: English comments, English plot names, updated report and logs

erdemkarabil committed 17 minutes ago

00a438b

Commits on Jun 3, 2026

docs: add inline English comments explaining architecture and design decisions

erdemkarabil committed 2 days ago

9dd5183

Commits on May 18, 2026

README: add GitHub repo link and venv setup note

erdemkarabil committed 3 weeks ago

6e57612

Update project report to Report.pdf and refresh README link

erdemkarabil committed 3 weeks ago

1408463

VoltOptimizer: CI&DL term project - source and docs

erdemkarabil committed 3 weeks ago

d5b0404

CIDL — Project 2

Page 22 / 22